

Group Design Report — Lumify LMS

Module: UFCF7S-30-2 — Systems Development
Project: Lumify — Learning Management System
Team: Five members

UWE ID	Name	Role
24070536	Shaamil	Backend & Database Architect
24066489	Vishal	Frontend Lead & UI/UX
24066475	Mohamed Hameez	Full-Stack Developer (Course & Content)
24066470	Lene	Full-Stack Developer (Assessment & QA)
24066483	Ali Raaish Rasheed	Team Lead

1. Executive Summary

Lumify is a web-based Learning Management System built as the capstone project for UFCF7S-30-2. It supports three roles: Student, Instructor, and Administrator, each with a distinct set of capabilities. Students browse and enrol in courses, track lesson progress, submit assignments, and view grades. Instructors create and publish course content, review submissions, and provide graded feedback. Administrators manage user accounts, oversee courses, and monitor platform activity.

The system was developed over four SCRUM sprints using a TypeScript-first monorepo architecture built on Nuxt 4, PostgreSQL, and Prisma 7. Authentication uses dual JWT tokens (access + refresh), all server-side routes enforce role-based access controls, and all user input is validated through Zod v4 schemas and XSS sanitisation. The test suite comprises 129 tests: 74 unit, 42 API integration, and 13 browser-based end-to-end all passing with zero TypeScript compilation errors under strict mode.

Beyond the core requirements, the system includes a learning streak gamification tracker, a course announcement system with student notification broadcasts, role-scoped direct messaging, and a branded loading screen with CSS animation.

2. Development Methodology

2.1 Agile SCRUM

Lumify was developed using SCRUM across four sprints. For a five-member team with evolving requirements over a single semester, the structure worked well: sprint reviews gave the team a regular checkpoint for feedback, and retrospectives surfaced problems early enough to address them..

Factor	Rationale
Team size	Five members — ideal for a single SCRUM team with balanced workloads
Requirement volatility	University project scope refined throughout the semester as features were validated against the specification
Iterative delivery	Each sprint produced a demonstrable increment, enabling early course correction
Risk management	Sprint reviews surfaced issues early; the risk register was updated continuously
Stakeholder alignment	Sprint demos ensured deliverables met assessment criteria at every milestone

2.2 Sprint Breakdown

The project was executed across **four sprints** of approximately two weeks each:

Sprint 1 — Foundation (Weeks 1–2)

Deliverable	Details
Project setup	Nuxt 4 initialised, PostgreSQL configured, Prisma schema with initial migration
Authentication system	Register, login, JWT access/refresh token management, Pinia auth store
Database schema	Core models (User, Course, Module, Lesson, Enrollment) and enums
UI shell	Auth layout, default layout, landing page, loading screen animation
Admin approval workflow	PENDING user registration with admin approve/reject mechanics

Sprint 2 — Course & Lesson Delivery (Weeks 3–4)

Deliverable	Details
Course catalogue	Public course listing with search, course detail page
Enrollment system	Student enrollment with admin approval pipeline

Deliverable	Details
Module/Lesson CRUD	Instructor-created modules and lessons with content, video URLs, and ordering
Lesson viewer	Student lesson page with content rendering, resources, prev/next navigation
Lesson completion tracking	LessonProgress upsert with completed timestamp
Student dashboard	Stats cards, enrolled courses with progress bars, streak tracker

Sprint 3 — Assignments, Grading & Communication (Weeks 5–6)

Deliverable	Details
Assignment system	Instructor-created assignments with due dates, max points, and DRAFT/PUBLISHED/CLOSED lifecycle
Submission workflow	Student submission with text content and file upload; multipart form handling
Grading workflow	Instructor submission review, numeric grading with feedback, notification to student
Messaging system	Direct messages between any two users with conversation view and recipient search
Notification system	Bell icon with unread badge, clickable notifications, mark-all-read
Instructor dashboard	Course management, student counts, pending submission counts

Sprint 4 — Admin Features, Testing & Polish (Weeks 7–8)

Deliverable	Details
Admin panel	User management (search, filter, approve, reject, deactivate), course oversight, activity log
Course announcements	Full CRUD API with role enforcement and notification broadcast to enrolled students
Rate limiting	IP-based in-memory rate limiter on authentication endpoints (login, register, forgot-password)
XSS sanitisation	stripHtml() applied to all user-provided text fields
Password reset	Forgot-password token generation (crypto-random, 1-hour expiry) and reset flow
Full-text search	pg_trgm GIN indexes on title, description, email, and name fields

Deliverable	Details
Seed data	Realistic demo data: 3 courses, 1 instructor, 2 students, 5 modules, 10 lessons, 5 assignments
TypeScript strict mode	Enabled "strict": true; resolved 12+ pre-existing type errors across 9 server files
Integration tests	42 API tests across 6 suites (auth, enrollment, assignments, lessons, notifications, file upload)
Unit tests	74 logic tests across 4 suites (middleware, JWT, validation, dashboard stats)
E2E tests	13 Playwright browser tests (auth flows, admin navigation, authorization guards)
CI/CD pipeline	GitHub Actions workflow with PostgreSQL service, migrations, seed, typecheck, Vitest, Playwright
Pre-commit hooks	Husky + lint-staged running nuxt typecheck on staged .ts/.vue files

2.3 Team Roles

Developer	Role	Responsibilities
Developer 1	Backend & Database Architect	Database design, Prisma schema, migrations, core backend logic, user management APIs
Developer 2	Frontend Lead & UI/UX	Overall look and feel, layouts, components, dashboards, responsive design, loading screen
Developer 3	Full-Stack (Course & Content)	End-to-end course catalogue, module/lesson CRUD, enrollment workflow, lesson viewer
Developer 4	Full-Stack (Assessment & QA)	Assignment submission/grading, file uploads, messaging, test suites (Vitest + Playwright)
Developer 5	Team Lead	Task assignment, pull request reviews, methodology documentation, CI/CD pipeline, risk assessment
All members	Collaborative	Sprint planning, daily standups, sprint reviews, retrospectives, code reviews

2.4 Version Control Strategy

The team adopted **GitHub Flow** with the following conventions:

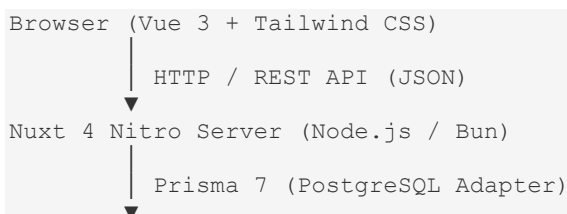
- Main branch (main): Production-ready code. All merges required a passing CI pipeline and at least one peer review.
- Feature branches: Created per sprint deliverable (e.g., User-Login/Auth-w-Dashboards, e2e-tests). Branches were short-lived and merged within the sprint window.
- Pull requests: All changes merged via PR with mandatory code review. The team lead performed final review before merging to main.
- Conventional commits: Messages followed the format type: description where type was one of feat, fix, test, ci, docs, chore, or refactor.

2.5 Tools & Practices

Category	Tool / Practice
Version control	Git + GitHub
Project management	GitHub Issues / Projects
CI/CD	GitHub Actions (PostgreSQL service, typecheck, migrations, seed, Vitest, Playwright)
Pre-commit hooks	Husky + lint-staged (nuxt typecheck)
Code quality	TypeScript strict mode, 2-space indent, single quotes, no semicolons
Unit testing	Vitest v4 — 74 logic tests
Integration testing	Vitest v4 + ofetch — 42 API tests
E2E testing	Playwright v1.60 — 13 browser tests
Total test coverage	129 tests, 0 failures, 0 type errors
Documentation	Markdown docs in /docs, Mermaid diagrams, PlantUML use-case

3. System Architecture

Lumify follows a **monolithic full-stack architecture** where server and client share a single Nuxt 4 codebase with clear separation of concerns. The browser renders Vue 3 components styled with Tailwind CSS v4. All data requests flow as RESTful HTTP calls to the Nitro server engine embedded within Nuxt, which uses Prisma 7 to query a PostgreSQL 14+ database.



3.1 Request Lifecycle

1. The browser issues an HTTP request to a Nuxt route.
2. Nuxt middleware (app/middleware/auth.ts and guest.ts) validates the user's session state from the Pinia auth store. Unauthenticated requests to protected routes are redirected to /login; role-mismatched requests receive a redirect to the appropriate dashboard.
3. API requests hit a Nitro server handler in server/api/, which extracts the JWT from the lms_token cookie or Authorization: Bearer header via the requireAuth utility.
4. The handler validates request bodies against Zod schemas, sanitises string inputs, and executes Prisma queries against PostgreSQL.
5. Responses are returned as JSON. Errors follow a consistent { statusCode, message } shape.

4. Technology Stack

Layer	Technology	Version	Rationale
Runtime	Bun	1.x	Fast package manager and runtime; drop-in Node.js replacement
Framework	Nuxt	4.x	SSR, file-based routing, Nitro server engine, auto-imports
Frontend	Vue 3 + Composition API	3.5	Reactive UI with <script setup> sugar
CSS	Tailwind CSS + Nuxt UI	v4 / 4.5	Utility-first styling; pre-built accessible components
Icons	Lucide Vue Next	1.x	Consistent iconography across 34+ pages
State	Pinia	3.x	Lightweight stores; auth persistence via useCookie
Validation	Zod	4.x	Schema-based runtime validation with TypeScript inference
Database	PostgreSQL	14+	ACID compliance, relational integrity, gin_trgm full-text search
ORM	Prisma + pg-adapter	7.x	Type-safe queries, migration system, connection pooling

Layer	Technology	Version	Rationale
Auth	jsonwebtoken + bcryptjs	9.x / 3.x	Stateless JWT auth; bcrypt at 12 salt rounds
Testing	Vitest + Playwright	4.x / 1.60	Unit + API integration + browser E2E
CI/CD	GitHub Actions	—	PostgreSQL service, migrations, seed, tests
Hooks	Husky + lint-staged	9.x / 17.x	Pre-commit typecheck on staged .ts/.vue files

5. Project Structure & Routing

The application follows a structured monorepo layout dividing responsibilities between frontend and backend:

```

lms-app/
├── app/                                # Frontend (Vue 3)
│   ├── app.vue                        # Root component
│   ├── components/common/            # LoadingScreen, AppToast
│   ├── composables/                  # useAppToast
│   ├── layouts/                      # landing, auth, dashboard, default
│   ├── middleware/                   # auth.ts, guest.ts
│   ├── pages/                        # 26 file-based routes
│   └── stores/                       # Pinia auth store
├── server/                            # Backend (Nitro)
│   ├── api/                          # 74+ route handlers
│   │   ├── admin/                   # 16 admin endpoints
│   │   ├── announcements/           # 4 CRUD endpoints
│   │   ├── auth/                    # 6 auth endpoints
│   │   ├── courses/                 # 10 course + lesson endpoints
│   │   ├── instructor/              # 13 instructor endpoints
│   │   ├── messages/                # 4 messaging endpoints
│   │   ├── notifications/           # 3 notification endpoints
│   │   ├── student/                 # 3 student dashboard endpoints
│   │   └── user/                    # 3 profile endpoints
│   └── utils/                        # jwt, password, auth, notify, rate-limit, sanitize
├── prisma/                            # Database
│   ├── schema.prisma                 # 15 models, 6 enums
│   ├── seed.ts                       # Demo data (5 users, 3 courses)
│   └── migrations/                   # 8 version-controlled migrations
├── lib/                              # Prisma client singleton
├── tests/                             # 10 test files (129 tests)
├── e2e/                              # Playwright browser tests (13 tests)
├── .github/workflows/                 # CI pipeline
└── docs/                             # Documentation + diagrams

```

5.1 Route Middleware

auth.ts — Redirects unauthenticated users to /login. Blocks students and instructors from /admin/* routes (redirect to /dashboard). Redirects administrators from /dashboard to /admin.

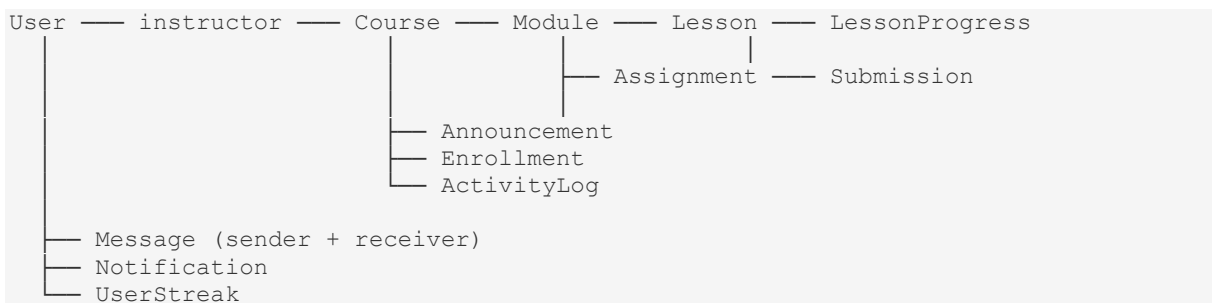
guest.ts — Redirects already-authenticated users away from /login and /register. Administrators go to /admin; all other roles go to /dashboard.

These middleware checks are layered on top of server-side authentication — they provide instant UX feedback but are never the sole enforcement mechanism.

6. Database Design

The schema comprises **15 models** and **6 enums**, anchored by a central User model.

6.1 Entity-Relationship Overview



6.2 Key Models

Model	Purpose	Key Fields
User	Central identity for all roles	email (unique), passwordHash, role (ADMIN/INSTRUCTOR/STUDENT), status (PENDING/ACTIVE/REJECTED), refreshToken, passwordResetToken
Course	A published or draft learning unit	title, description, code (unique), color, duration, isPublished, instructorId
Module	Ordered section within a course	title, description, order, courseId

Model	Purpose	Key Fields
Lesson	Content unit within a module	title, content, videoUrl, duration, order
LessonProgress	Per-student lesson completion	userId, lessonId, completed, completedAt (unique pair)
Assignment	Assessable task within a module	title, description, dueDate, maxPoints, status (DRAFT/PUBLISHED/CLOSED)
Submission	Student response to an assignment	content, fileUrl, grade, feedback, status (PENDING/SUBMITTED/LATE/GRADED) (unique pair)
Enrollment	Student-course relationship	userId, courseId, status (PENDING/ACTIVE/REJECTED/COMPLETED/DROPPED) (unique pair)
Message	Direct message between two users	content, senderId, receiverId, read
Notification	System alert for a user	title, message, type, read, link (cascade delete)
Announcement	Course-wide broadcast	title, content, courseId, authorId
ActivityLog	Audit trail of system events	type (ActivityLogType), description, userId, courseId (indexed)
UserStreak	Gamification tracking	currentStreak, longestStreak, lastActiveDate (one-to-one with User)

6.3 Enums

Enum	Values
Role	ADMIN, INSTRUCTOR, STUDENT
UserStatus	PENDING, ACTIVE, REJECTED
EnrollmentStatus	PENDING, ACTIVE, REJECTED, COMPLETED, DROPPED
AssignmentStatus	DRAFT, PUBLISHED, CLOSED

Enum	Values
SubmissionStatus	PENDING, SUBMITTED, LATE, GRADED
ActivityLogType	COMPLETED_MODULE, SUBMITTED_ASSIGNMENT, STARTED_COURSE, RECEIVED_FEEDBACK, COMPLETED_LESSON, ENROLLED_COURSE, ACCOUNT_APPROVED

6.4 Constraints & Indexes

- Unique constraints: User.email, Course.code, CourseCategory.name, Enrollment[userId, courseId], Submission[userId, assignmentId], LessonProgress[userId, lessonId], UserStreak.userId
- Cascade deletes: Module → Lesson → LessonProgress; Module → Assignment → Submission; Course → Module/Enrollment/Announcement; Lesson → Resource; User → Notification
- Full-text search: pg_trgm GIN indexes on Course.title, Course.description, User.email, and a composite (firstName || ' ' || lastName) expression index on User
- Performance indexes: ActivityLog(userId), ActivityLog(createdAt), Notification(userId), lowercase Course.code

6.5 Assignment Lifecycle

Assignments follow a strict one-directional state machine:



- DRAFT: Instructor is preparing the assignment; not visible to students.
- PUBLISHED: Students can submit; the server automatically flags submissions past dueDate as LATE.
- CLOSED: No further submissions accepted; existing submissions retain their grades.
- Once transitions are made, they cannot be reversed.

7. Authentication & Authorisation

7.1 Dual-Token Strategy

The system uses a dual-token JWT strategy:

Token	TTL	Storage	Purpose
Access Token	15 minutes	lms_token cookie (SameSite: lax)	Authenticates API requests; short-lived to limit exposure

Token	TTL	Storage	Purpose
Refresh Token	7 days	refresh_token cookie (httpOnly, SameSite: lax)	Issues new access tokens without re-login; stored as bcrypt hash in User.refreshToken

Login flow:

6. Server validates credentials against `bcrypt.compare()`.
7. If valid, generates an access token (signed with `JWT_ACCESS_SECRET`) and a refresh token (signed with `JWT_REFRESH_SECRET`).
8. The refresh token is bcrypt-hashed and persisted to `User.refreshToken`; the raw refresh token is set as an `httpOnly` cookie.
9. The access token and user object are returned to the client.
10. The Pinia auth store persists both in cookies via Nuxt's `useCookie` composable.

Token refresh flow:

1. On a 401 response, the `fetchWithAuth` wrapper calls `POST /api/auth/refresh`.
2. The server validates the `refresh_token` cookie, verifies the token against the stored hash, and issues a new access token.
3. If refresh fails, the user is logged out and redirected to `/login`.

7.2 Server-Side Authorisation

Every protected API route applies one of these guards:

Guard	Source	Behaviour
<code>requireAuth()</code>	<code>server/utils/auth.ts</code>	Extracts JWT from <code>lms_token</code> cookie, verifies it, returns payload. Throws 401 if absent/invalid.
<code>requireAdmin()</code>	<code>server/utils/auth.ts</code>	Calls <code>requireAuth()</code> then checks <code>payload.role === 'ADMIN'</code> . Throws 403 on mismatch.
<code>requireInstructor()</code>	<code>server/utils/auth.ts</code>	Same pattern for INSTRUCTOR role.
<code>requireStudent()</code>	<code>server/utils/auth.ts</code>	Same pattern for STUDENT role.
<code>requireRole()</code>	<code>server/utils/requireAuth.ts</code>	Accepts multiple allowed roles (e.g., <code>requireRole(event, 'INSTRUCTOR', 'ADMIN')</code>).

All 25+ admin endpoints are behind `requireAdmin()`. Instructor-specific endpoints additionally verify course ownership via `course.instructorId === user.userId`. Student endpoints verify active enrollment before returning course content.

7.3 Client-Side Route Guards

Nuxt middleware (auth.ts and guest.ts) reads the Pinia auth store on every navigation and provides instant UX feedback, but all sensitive data and state-changing operations are protected server-side — the middleware is a convenience layer, not a security boundary.

8. API Design

The system exposes a RESTful API grouped into eleven resource areas:

8.1 Authentication (/api/auth/)

All endpoints are publicly accessible.

Method	Endpoint	Description
POST	/auth/register	Create a new account (status: PENDING)
POST	/auth/login	Authenticate, receive access token + user
POST	/auth/refresh	Exchange refresh_token cookie for new access token
POST	/auth/logout	Clear refresh_token cookie + database record
POST	/auth/forgot-password	Generate 32-byte hex reset token (1-hour expiry)
POST	/auth/reset-password	Consume token, set new password

8.2 Courses (/api/courses/)

Method	Endpoint	Auth	Description
GET	/courses	Public	List published courses (supports ?search=)
GET	/courses/:id	Student/Instructor/Admin	Course detail with modules, lessons, assignments, progress
POST	/courses/:id/enroll	Student	Create PENDING enrollment
GET	/courses/:id/lessons/:lid	Student/Instructor/Admin	Lesson detail with progress,

Method	Endpoint	Auth	Description
POST	/courses/:id/lessons/:lid/complete	Student	resources, prev/next Mark lesson complete (upserts LessonProgress)
POST	/courses/:id/assignments/:aid/submit	Student	Submit assignment (multipart file + text, 10MB limit)

8.3 Instructor (/api/instructor/)

Method	Endpoint	Auth	Description
GET	/instructor/dashboard	Instructor	Course stats, student counts, pending reviews
GET	/instructor/courses	Instructor	Courses owned by authenticated instructor
GET	/instructor/courses/:id	Instructor	Full course management detail
POST	/instructor/courses/:id/publish	Instructor	Toggle course publication
POST	/instructor/courses/:id/modules	Instructor	Create module (auto-increments order)
PUT	/instructor/modules/:id	Instructor	Update module
DELETE	/instructor/modules/:id	Instructor	Delete module (cascade)
POST	/instructor/modules/:id/lessons	Instructor	Create lesson
PUT	/instructor/lessons/:id	Instructor	Update lesson
DELETE	/instructor/lessons/:id	Instructor	Delete lesson
POST	/instructor/modules/:id/assignments	Instructor	Create assignment (DRAFT or PUBLISHED)
PUT	/instructor/assignments/:id	Instructor	Update assignment (enforces status transitions)
DELETE	/instructor/assignments/:id	Instructor	Delete assignment
GET	/instructor/assignments/:id/submissions	Instructor	All submissions for an assignment

Method	Endpoint	Auth	Description
POST	/instructor/submissions/:id/grade	Instructor	Grade submission (0–maxPoints, feedback)

8.4 Admin (/api/admin/)

All endpoints require Admin JWT.

Method	Endpoint	Description
GET	/admin/stats	System-wide statistics
GET	/admin/users	User directory (?role=, ?status=, ?search=)
GET	/admin/pending-users	Pending registrations
GET	/admin/activity	Unified activity feed (25 events)
POST	/admin/users/:id/approve	Approve pending user (sets ACTIVE, notifies)
POST	/admin/users/:id/reject	Reject pending user (sets REJECTED)
POST	/admin/users/:id/deactivate	Deactivate active user
GET	/admin/users/:id/enrollments	User's enrollment history
GET	/admin/users/:id/courses	Instructor's assigned courses
GET	/admin/courses	All courses with instructor + enrollment counts
POST	/admin/courses	Create course (auto-generates code, optional instructor)
DELETE	/admin/courses/:id	Delete course + enrollments + announcements
POST	/admin/courses/:id/assign-instructor	Assign or reassign instructor
GET	/admin/enrollments	All PENDING enrollments
POST	/admin/enrollments/:id/approve	Approve enrollment + notify student
POST	/admin/enrollments/:id/reject	Reject enrollment + notify student

8.5 Announcements (/api/announcements/)

Method	Endpoint	Auth	Description
GET	/announcements	Any	List all announcements (?courseId=filter)
POST	/announcements	Admin/Instructor	Post announcement (sanitises input, notifies enrolled students)

Method	Endpoint	Auth	Description
PUT	/announcements/:id	Author/Admin	Edit announcement
DELETE	/announcements/:id	Author/Admin	Delete announcement

8.6 Error Handling

All errors follow a consistent shape: { statusCode, message }

- 400 — Validation failure (Zod schema rejected input)
- 401 — Missing, expired, or invalid JWT
- 403 — Insufficient role or ownership check failure
- 404 — Resource not found
- 409 — Conflict (duplicate submission, already-graded, duplicate registration)
- 429 — Rate limit exceeded (production only)

9. Frontend Architecture

9.1 Layouts

Four layout contexts provide consistent navigation and chrome:

Layout	Used By	Navigation
Landing	Homepage (/)	Top nav: Lumify logo, Courses, Login, Register
Auth	/login, /register, /forgot-password, /reset-password	Left: branding + benefit bullets. Right: form panel
Dashboard	All authenticated pages	Full sidebar (role-aware), top bar (search, notifications, user chip)
Default	Fallback	Simple top nav with role-aware links

9.2 State Management

A single Pinia store (useAuthStore) manages authentication state:

- user — The authenticated user object (id, email, firstName, lastName, role)
- accessToken — Short-lived JWT persisted in lms_token cookie (15-minute maxAge)
- isLoggedIn — Derived boolean from userCookie
- isAdmin — Derived boolean from user.role === 'ADMIN'
- fetchWithAuth<T>() — Generic wrapper around ofetch that auto-refreshes on 401 and logs out on failure

- `setAuth()`, `logout()`, `refreshToken()` — Action methods

All cookies use `SameSite: lax` for CSRF protection and are scoped to the application path.

9.3 Notifications

- Polls `GET /api/notifications` on mount (for students/instructors) or `GET /api/admin/stats` (for admins, showing synthetic pending-approval notifications).
- Displays an unread count badge.
- Each notification is clickable with a deep link (e.g., `/courses/5` for an enrollment approval, `/assignments/12` for a grade).
- Mark all read clears the badge via `POST /api/notifications/read-all`.

9.4 Responsive Design

Tailwind CSS breakpoints (`sm`, `md`, `lg`) control layout adaptation. The sidebar collapses on small screens. Course grids switch from 3-column to 2-column to single-column based on viewport width. Tables gain horizontal scrolling on narrow screens.

10. Testing Strategy & Results

10.1 Approach

The test suite is built on **Vitest v4** (unit + API integration) and **Playwright v1.60** (browser E2E), executed via Bun. Testing follows a multi-tier strategy:

Tier	Framework	Scope	Count	Runtime
Unit	Vitest	Pure-logic functions isolated from infrastructure	74 assertions	< 1s
Integration	Vitest + ofetch	Full API workflows against a running server	42 tests	~22s
End-to-End	Playwright (Chromium)	Browser-based user journeys	13 tests	~19s
Type checking	nuxt typecheck	TypeScript strict mode across all files	0 errors	—

10.2 Unit Tests (74 assertions, 4 suites)

Unit tests extract core business logic into pure functions tested in isolation with no database or server dependency:

Suite	File	Coverage
Middleware	auth.middleware.test.ts	14 tests — every role × route combination for auth + guest middleware
JWT Verification	requireAuth.test.ts	8 tests — Bearer header extraction, cookie fallback, expired/invalid/malformed tokens
Assignment Validation	assignments.validation.test.ts	33 tests — creation rules (title length, maxPoints range, dueDate format), status transitions (DRAFT→PUBLISHED→CLOSED), late detection, submission content requirements, grade capping, and already-graded guard
Dashboard Stats	dashboard.stats.test.ts	12 tests — study hours calculation (0 lessons, 30×60min=30h, null durations), deadline urgency (within 48h, exactly 48h, 3+ days, past deadline), activity feed tag normalisation

10.3 Integration Tests (42 tests, 6 suites)

Integration tests make real HTTP requests against a development server and database:

Suite	Coverage
Auth (auth.test.ts)	T1.1–T1.6: Registration → PENDING block → admin approval → login with JWT; T5.1: Rate limiting (production only); T6.1–T6.5: Password reset flow
Enrollment (enrollment.test.ts)	T2.1–T2.6: Full enrollment workflow (register → approve → login → enroll → approve → dashboard → course access); T4.1–T4.6: Security — unauthenticated and role-mismatched admin access
Assignments (assignment.test.ts)	T3.1–T3.4: Submission → instructor review → grading → resubmission block
Lessons (lessons.test.ts)	T7.1–T7.5: Mark complete → progress reflected → idempotent → unauthenticated block → lesson detail
Notifications (notifications.test.ts)	T8.1–T8.5: Enrollment notification → unread count → mark-all-read → grade notification → unauthenticated block
File Upload (file-upload.test.ts)	T9.1–T9.4: Text-only submission → file URL → multipart FormData → empty submission rejection

10.4 End-to-End Tests (13 tests, 1 suite)

Playwright tests simulate real browser interactions in Chromium headless:

Suite	Coverage
Auth flow	Login page render, valid admin login, invalid credentials error, register page render, landing page render
Admin navigation	Dashboard, approvals, students, announcements, activity log
Authorization guards	/admin → redirect to /login, /admin/approvals → redirect to /login, /courses accessible without auth

10.5 Bugs Found & Fixed During Testing

#	Severity	Description	Fix
BUG-01	Critical	Seed upsert only updated status, not passwordHash — all seeded users locked out	Updated upsert clause
BUG-02	High	Full-text search migration had invalid gin_trgm_ops SQL	Wrapped expression in parentheses
BUG-03	High	Playwright clicks intercepted by 2-second loading screen overlay (z-[9999])	Added waitForTimeout(2500) before form interaction
BUG-04	Medium	New integration tests used admin token for instructor-only endpoints (403)	Created dedicated instructor accounts + assign-instructor
BUG-05	Medium	Playwright selectors expected 'Welcome back' but page shows 'Sign in to your account'	Updated selectors to match auth layout defaults

10.6 Results Summary

Metric	Value
Total tests	129
Passed	129
Failed	0
Skipped	0
TypeScript errors	0 (strict mode, 123 server + frontend files)

11. UML & System Modelling

All five required diagrams have been produced and rendered to PNG for submission. Mermaid and PlantUML source files are included in the repository for reproducibility.

Diagram	Format	Location	Description
Use Case	PlantUML → PNG	docs/diagrams/use-case/	32 use cases across 7 functional areas, 4 actors
Class Diagram	Mermaid → PNG	docs/diagrams/class-diagram/	15 models, 6 enums, all relationships and multiplicities
Sequence Diagrams	Mermaid → PNG	docs/diagrams/sequence-diagrams/	4 sequences: Login, Enrollment + Lesson Completion, Assignment Grading, Announcement Posting
Activity Diagrams	Mermaid → PNG	docs/diagrams/activity-diagrams/	4 activities: Registration & Approval, Course Content Management, Assignment Grading, Posting Announcements
ER Diagram	Mermaid → PNG	docs/diagrams/ER/	Complete entity-relationship model with PKs, FKs, enums

All diagrams accurately reflect the implemented system — not an aspirational design.

12. Non-Functional Requirements

Performance

- Page load under 3 seconds on standard broadband (Nuxt SSR + Vite bundling)
- API response time under 500ms for typical queries (Prisma selective field inclusion)
- Static assets served via Nuxt's built-in optimisation pipeline

Security

- JWT dual-token authentication (15-minute access, 7-day refresh)
- Refresh tokens stored as bcrypt hashes — not reversible
- All 25+ admin endpoints and instructor endpoints behind server-side authentication
- Password hashing with bcrypt (12 salt rounds)
- HTML tag stripping on all user inputs via `stripHtml()` + `sanitizeString()`
- File upload validation: MIME type whitelist (8 types), 10MB size limit, magic byte verification

- IP-based rate limiting on auth endpoints: 5 req/min login, 3 req/min register/forgot-password

Reliability

- PostgreSQL ACID compliance for data integrity
- Unique constraints on email, enrollment pairs, submission pairs, lesson progress pairs
- Cascade deletes for dependent records (Course → Modules → Lessons → Progress, etc.)

Usability

- Role-based navigation with labelled sections
- Consistent UI patterns via Nuxt UI component library and Tailwind CSS v4
- Toast notifications for success/error feedback (stacked, auto-dismiss after 4s)
- Loading spinners and empty states on all data-driven pages
- Responsive design supporting desktop and mobile

Maintainability

- Modular code structure: server/api/ routes, server/utils/ utilities, app/composables/, app/stores/
- TypeScript throughout with "strict": true enforced
- Prisma 7 ORM for type-safe database operations
- Git version control with conventional commit messages (feat:, fix:, test:, ci:, docs:)
- Pre-commit hooks running nuxt typecheck on staged files

Data Protection

- Passwords never stored in plaintext (bcrypt hashed at 12 rounds)
- JWT secrets stored in .env (gitignored); .env.example uses placeholders
- Refresh tokens hashed before database storage
- User data accessible only via authenticated, authorised API calls

Accessibility

- Semantic HTML elements (<nav>, <main>, <aside>, <header>, <form>, <label>)
- Form inputs associated with <label> elements via for/id
- Dashboard layout includes aria-label, aria-current, role attributes
- "Skip to content" link at the top of the dashboard
- Responsive design supports zoom and text scaling

Scalability

- PostgreSQL connection pooling via Prisma adapter
- Stateless JWT authentication enables horizontal scaling
- File uploads stored on local filesystem (migratable to S3-compatible object storage)

13. Security Considerations

The system implements defence-in-depth with controls at multiple layers:

Layer	Control
Transport	HTTPS recommended in production; SameSite: lax cookies
Authentication	Dual JWT tokens with short-lived access + hashed refresh
Authorisation	Server-side role checks on every protected route; instructor ownership verification
Input validation	Zod v4 schemas on all request bodies; stripHtml() XSS sanitisation on string fields
File uploads	MIME whitelist (8 types), 10MB limit, magic byte verification, extension whitelist
Rate limiting	IP-based in-memory rate limiter on auth endpoints (prod-only)
Secrets	.env gitignored; .env.example with placeholders only
Passwords	bcrypt at 12 salt rounds; never logged or returned in API responses
Session	Refresh tokens deletable server-side via logout; access tokens expire in 15 min
CSRF	JWT in SameSite: lax cookies; no state-changing GET endpoints

14. Ethical & Legal Compliance

Data Protection (DPA 2018 / UK GDPR Principles)

- Lawfulness, fairness, and transparency: Users are informed that their data (email, name, role, course activity) is stored for the purpose of operating the LMS. Registration requires explicit consent via form submission.
- Purpose limitation: Data is used solely for LMS functionality — course delivery, assignment grading, and platform administration.
- Data minimisation: Only data essential to core functionality is collected. No analytics, tracking cookies, or third-party integrations.
- Accuracy: Users can edit their profile (name, bio) and change their password at any time.

- Storage limitation: Refresh tokens are cleared on logout. Password reset tokens expire after 1 hour.
- Integrity and confidentiality: All passwords are bcrypt-hashed. JWT secrets are environment-variable-only. Database access is gated behind Prisma's connection pool.

Academic Integrity

- Assignment submission tracking: The system records submission timestamps and prevents resubmission after grading, supporting academic honesty.
- Late detection: Automatic LATE flag on submissions past the due date provides objective, auditable enforcement of deadlines.
- Audit trail: ActivityLog records all significant system events (registrations, enrollments, submissions, approvals) with timestamps and user attribution.

Accessibility & Inclusivity

- Semantic HTML, labelled form inputs, and responsive design support users of assistive technologies.
- The "Skip to content" link on the dashboard improves keyboard navigation.
- Further ARIA coverage is identified as ongoing work (see Risk R14 in the risk register).

Professional Code of Conduct

- The team followed the BCS Code of Conduct throughout development.
- All code was peer-reviewed via GitHub pull requests.
- Conventional commits provide an auditable development history.
- No sensitive data (passwords, tokens) was committed to the repository at any point.

15. Innovation & Improvement Features

Lumify incorporates several features that differentiate it from a standard Moodle deployment:

Learning Streak Tracker (Gamification)

Tracks consecutive days of lesson completion via the UserStreak model. The student dashboard shows a 7-day visual calendar with checkmark indicators and milestone messages that update based on streak length. The streak resets if no lesson is completed on the current or previous day.

Course Announcements with Notification Broadcast

Instructors and admins post announcements to specific courses. The backend queries enrolled students and creates a Notification record for each one. Students receive bell alerts with direct links to the relevant course. Announcements can be deleted after publication.

In-App Direct Messaging

Any user can message any other user. A role-scoped recipient directory filters available contacts. The conversation view includes auto-scroll, colour-coded messages, and a new message modal with search.

Professional Branded Experience

A custom CSS-animated loading screen with a dot-grid background, logo entrance, progress bar, and pulse ring. The dark-themed sidebar uses gradient user avatars throughout, with Nuxt UI components handling form elements.

16. Known Limitations & Future Work

Limitation	Severity	Planned Resolution
No quiz/assessment module (MCQ, auto-grading)	Medium	Schema designed in assessment-feature-spec.md; targeted for v2.0
Announcements only visible to students via notifications — no dedicated announcements view on course pages	Low	Add announcements section to course detail page
No instructor-facing announcements page in sidebar nav (instructors use admin page)	Low	Add /instructor/announcements route
Rate limiting is in-memory only — resets on server restart	Low	Migrate to Redis or PostgreSQL-backed store for production
File uploads stored on local filesystem — not horizontally scalable	Low	Migrate to S3-compatible object storage for multi-instance deployments
No email integration for password reset (token returned in API response for development)	Medium	Integrate Nodemailer or SendGrid for production deployment
CourseCategory and Resource models exist in schema but lack full API/UI integration	Low	Build category filtering and resource upload/management features
WCAG ARIA coverage is partial — more attributes needed for screen reader compatibility	Low	Systematic ARIA audit across all components

17. Conclusion

Lumify is a working Learning Management System that covers all requirements in the UFCF7S-30-2 specification:

- Four SCRUM sprints with clear deliverables at each stage
 - 15 database models, 6 enums, proper constraints, cascade deletes, and performance indexes
 - Dual JWT authentication, bcrypt hashing, server-side role enforcement, and rate limiting
 - 74+ API endpoints covering all functional requirements
 - Responsive frontend with role-aware navigation, loading states, and empty-state handling
 - 129 tests (unit, integration, and E2E), all passing, zero TypeScript errors under strict mode
 - Documentation including this design report, system documentation, a user guide, a methodology document, a 16-item risk register, and a test plan
- Version control with conventional commits, CI/CD pipeline automation, pre-commit quality gates, and peer-reviewed pull

requests were used throughout. All deliverables from the module brief are included..

End of Report